Network-Level Ad Blocker Using Pi-hole: A Self-Hosted DNS Filtering Solution

John Callaghan

May 12, 2026

Abstract

This report documents the design and deployment of a self-hosted, network-wide ad blocking solution using a Raspberry Pi running Pi-hole. The system intercepts DNS queries across all devices on a home network, blocking ads and trackers before they are loaded. The project demonstrates practical skills in Linux system administration, networking (DNS/DHCP), debugging, and self-hosted infrastructure.

1 Overview

A network-level ad blocker was designed and deployed using a Raspberry Pi. The system acts as a DNS sinkhole: all devices on the local network send DNS queries to the Pi-hole instance, which filters out requests to known ad and tracker domains. This eliminates the need to install ad-blocking software on individual devices such as smart TVs, phones, laptops, and gaming consoles.

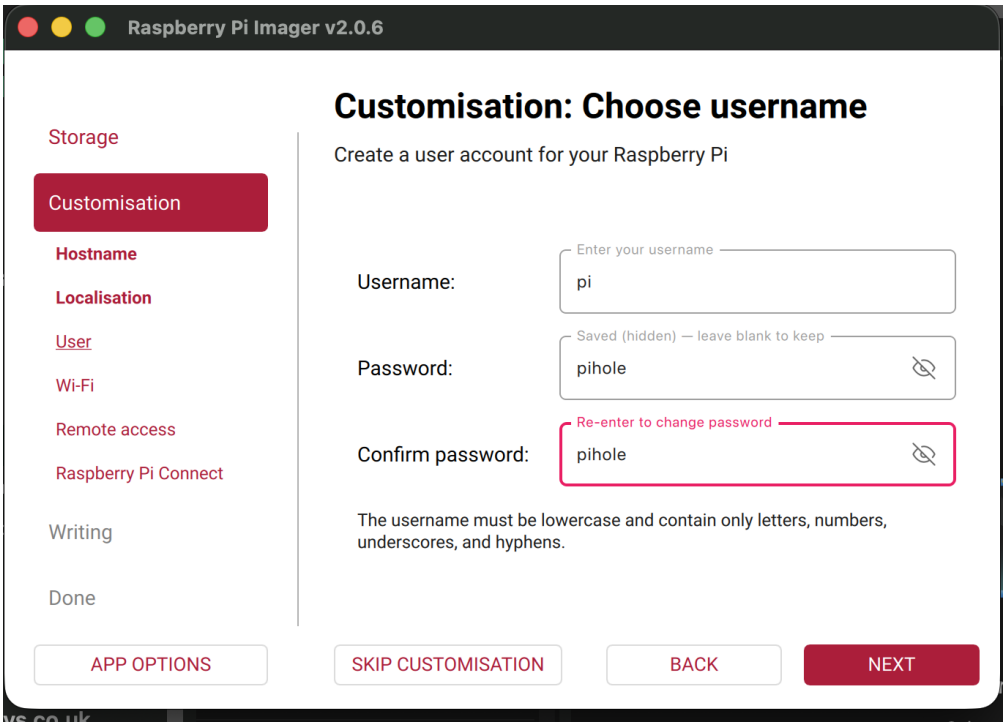

Figure 1: Pi-hole customization before completing installation.

2 Objective

The primary objectives were:

- Block advertisements and trackers across all devices on a home network without per-device configuration.
- Gain hands-on experience with Linux administration, networking (DNS/DHCP), and self-hosted infrastructure.
- Develop debugging skills by resolving real-world issues (static IP configuration, IP conflicts, router access).

3 Technology Stack

- **Hardware**: Raspberry Pi (ARMv6, 512MB RAM)
- **Operating System**: Raspberry Pi OS Lite (headless, no desktop)
- **Software**: Pi-hole, NetworkManager (`nmcli`)
- **Protocols**: DNS, DHCP
- **Access**: SSH from macOS

4 Implementation Steps

4.1 Operating System Setup

Raspberry Pi OS Lite was flashed to a microSD card using the Raspberry Pi Imager. During this step, the hostname was configured and SSH was enabled, allowing headless access from the first boot.

4.2 Initial Access and Updates

After connecting the Raspberry Pi to the network, remote access was established via SSH. System packages were updated to ensure a stable base:

```
sudo apt update && sudo apt upgrade -y
```

4.3 Static IP Configuration

A static IP address was required for the Pi-hole to act as a consistent DNS server. Initially, changes were made to the legacy `dhcpcd.conf` file, but they had no effect. Investigation revealed that the OS image had switched to NetworkManager instead of `dhcpcd`. The configuration was successfully applied using `nmcli`:

```
nmcli con mod  Wired␣connection␣1  ipv4.addresses <static-ip>/24
nmcli con mod  Wired␣connection␣1  ipv4.gateway <gateway-ip>
nmcli con mod  Wired␣connection␣1  ipv4.dns  1.1.1.1
nmcli con mod  Wired␣connection␣1  ipv4.method manual
nmcli con up  Wired␣connection␣1
```

(Actual IP and gateway values are omitted for security.)

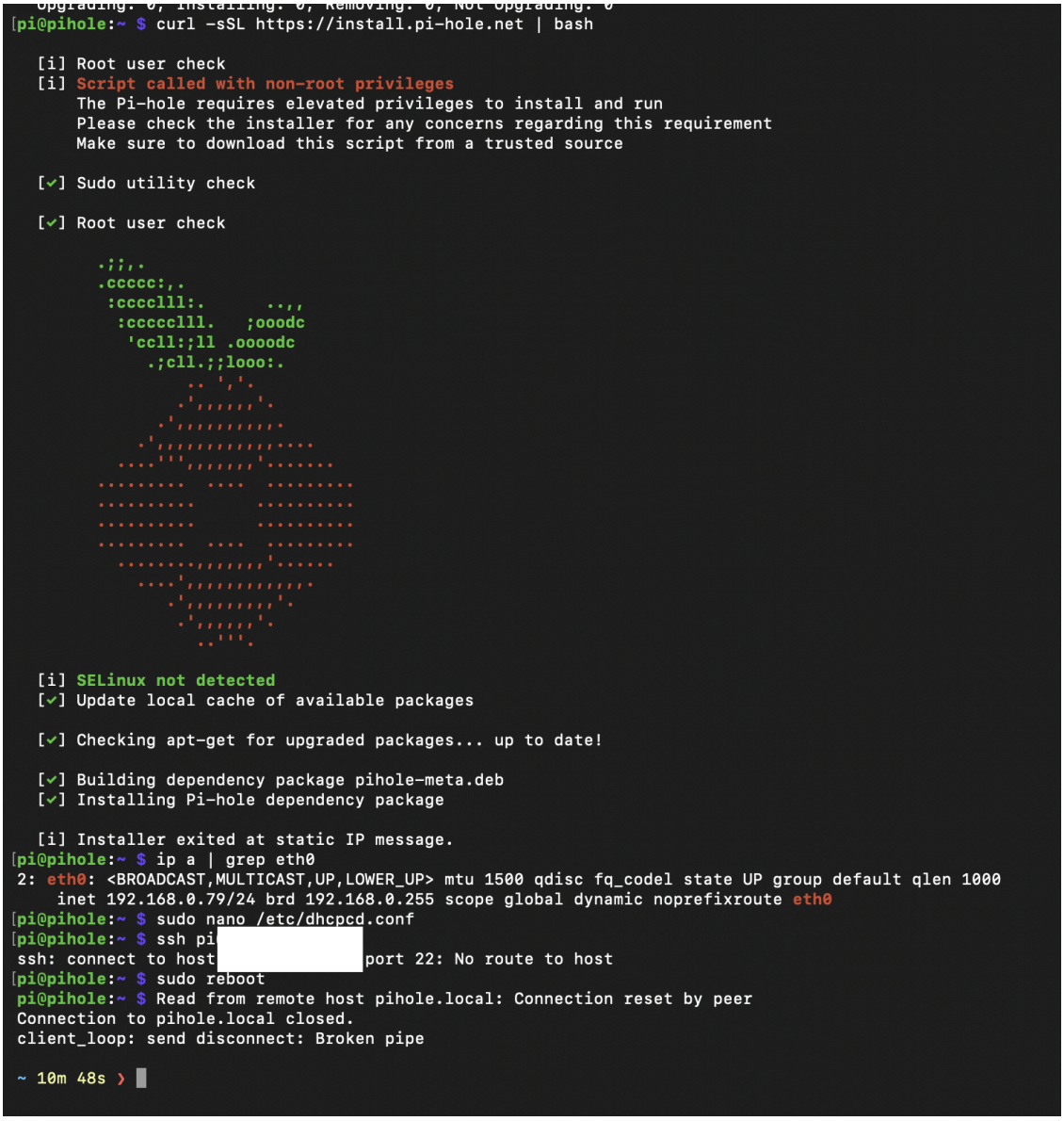

Figure 2: SSH session into the Raspberry Pi showing the Pi logo and terminal access.

4.4 Installing Pi-hole

Pi-hole was installed using the official automated installer:

```
curl -sSL https://install.pi-hole.net | bash
```

During the installation, Cloudflare DNS (1.1.1.1) was selected as the upstream resolver. The installer automatically set up a local web server (lighttpd) for the admin dashboard.

4.5 Router Configuration

The home router's DHCP settings were modified to advertise the Pi-hole's static IP as the primary DNS server for all clients. After saving the changes and renewing DHCP leases (or rebooting devices), every device on the network began sending DNS queries through Pi-hole without any per-device configuration.

Figure 3: Final hardware setup: Raspberry Pi connected to the network, ready for 24/7 operation.

5 Challenges & Problem Solving

5.1 Static IP Not Applying

The initial attempt to set a static IP via `/etc/dhcpcd.conf` failed because the operating system had migrated to NetworkManager. This was diagnosed by checking running processes and network configuration files. Switching to `nmcli` resolved the issue.

5.2 IP Address Conflict

An existing DHCP reservation for the target IP address on the router caused a conflict. Using `arp -a` confirmed that the Raspberry Pi had already claimed the IP successfully despite the stale reservation – the conflict was benign, but understanding it avoided unnecessary troubleshooting.

5.3 Router Administration

The router's admin credentials were temporarily unavailable. Instead of changing DHCP reservations, the static IP was configured directly on the Pi, which proved sufficient for the project.

6 Outcome

- Pi-hole is live and serves as the DNS resolver for the entire home network.
- All connected devices automatically route DNS queries through Pi-hole with zero per-device configuration.
- Ad blocking and query logging are visible in real time via the Pi-hole web dashboard.
- The solution blocks ads and trackers on devices that traditionally lack ad-blocking software, such as smart TVs and IoT devices.

7 Skills Demonstrated

- Linux command-line interface (CLI) proficiency
- SSH remote access and system management
- Networking fundamentals (DNS, DHCP, static IP assignment)
- NetworkManager configuration using `nmcli`
- Debugging real-world configuration issues
- Self-hosted infrastructure deployment
- Raspberry Pi hardware and OS setup

8 Conclusion

This project successfully deployed a network-wide ad blocker using Pi-hole on a Raspberry Pi. It eliminated intrusive advertisements across all devices in the home, improved privacy by blocking trackers, and provided valuable hands-on experience with Linux system administration, networking, and troubleshooting. The solution is lightweight, low-cost, and has been running reliably without intervention.